

3-4 · DuckDB를 이용한 데이터 분석 · 정리·회고 0.5H

정리·회고

분석 문법을 한 장으로, 3-5 자료 모으기

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

- ① DuckDB 분석 문법 열한 가지를 표로 한눈에 본다
- ② SQL 이 나왔던 것·Pandas 가 나왔던 것을 하나씩 적는다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 3-5 하이브리드 가공에 필요한 orders.csv 를 확인한다

「결과가 미묘하게 다른」 지점을 적어 두는 것이 3-5 의 재료입니다

DuckDB 분석 문법 — 집계와 윈도우

하려는 일	문법
조건부 집계	count(*) FILTER (WHERE 조건)
날짜 자르기	date_trunc('day' 'week' 'month', ts)
그룹별 순위	rank() OVER (PARTITION BY ... ORDER BY ...)
순위 거르기	QUALIFY rnk <= 3 (DuckDB 전용)
누적	sum(x) OVER (ORDER BY d)
이동 평균	avg(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

QUALIFY·GROUP BY ALL·EXCLUDE 는 DuckDB 전용 — 옮겨 다닐 질의라면 표준 문법으로

DuckDB 분석 문법 — 구조와 왕복

하려는 일	문법
단계 나누기	WITH a AS (...), b AS (...)
알아서 묶기	GROUP BY ALL (DuckDB 전용)
열 빼고 전부	SELECT * EXCLUDE (열) (DuckDB 전용)
DataFrame 으로	.df() — 반드시 줄인 다음
DataFrame 질의	FROM 파이썬변수

.df() 는 반드시 줄인 다음 — 집계 전 원본에 부르면 100만 행이 통째로 올라온다

3-5 설계에 쓸 자료 — 각각 하나씩 적어 두세요

01

SQL 이 나왔던 것

예 — 100만 행
사용자별 집계

02

Pandas 가 나왔던 것

예 — 분위수 경계로
등급 나누기

03

둘 다 되는 것

무엇을 골랐고
왜 골랐나

04

미묘하게 다른 지점

예 — 이동 평균의
앞쪽 처리

3-5에서는 「무엇을 SQL 로, 무엇을 Pandas 로」를 스스로 정해 파이프라인을 만듭니다

여덟 개를 스스로 확인해 보세요

- 1 2-2 의 CTE·Window 문법이 DuckDB 에서 그대로 통했다
- 2 Parquet·CSV 끼리 JOIN 하고 Pandas 결과와 교차 검증했다
- 3 CTE 로 단계를 나누고 중간 결과를 따로 확인했다
- 4 rank + QUALIFY 로 그룹별 상위 N 을 뽑고 CTE 로도 만들었다
- 5 rolling 과 윈도우 프레임의 앞쪽 처리 차이를 확인했다
- 6 SQL → Pandas → SQL 왕복으로 세그먼트 분석을 만들었다
- 7 줄이기 전·후 .df() 의 시간·메모리 차이를 실측했다
- 8 WHERE 윈도우 함수 실패와 눈대중 경계의 전원 S 를 직접 봤다

증상으로 찾는 원인과 해결

증상	원인	해결
Binder Error — WHERE 에 원도 함수	WHERE 는 원도 함수 계산 전에 행을 거른다	QUALIFY 로 거르거나 CTE 로 감싸고 WHERE
rank 결과가 실행마다 다르게 보임	동률 안의 출력 순서는 보장되지 않는다	정렬 키에 고유 열 추가 rank 와 row_number 비교
.df() 메모리 부족 또는 매우 느림	집계 전 원본을 통째로 변환	집계·필터·LIMIT 로 줄인 뒤 .df()
QUALIFY 가 다른 DB 에서 문법 오류	표준 SQL 이 아닌 DuckDB 계열 확장	웁길 질의는 CTE 로 감싸고 WHERE

첫 줄은 1.5H 5단계에서 일부러 낸 오류 · 변환은 항상 「작아진 다음」

자주 나오는 질문 네 가지

- 1 SQL 과 Pandas 중 하나만 잘하면?**
실무는 둘 다 — 대용량 도구 대부분이 SQL 인터페이스(5주차 SparkSQL·4주차 Trino)
- 2 FILTER (WHERE ...) 가 안 되는 DB 도?**
MySQL 에는 없다 — SUM(CASE WHEN 조건 THEN 1 ELSE 0 END) 으로
- 3 왕복할 때마다 복사되나요?**
.df() 는 복사된다 — 그래서 줄인 다음에. FROM 변수는 대개 복사 없이 참조
- 4 질의가 느린데 어디를 봐야?**
EXPLAIN 을 앞에 붙이면 실행 계획 — 대개 읽는 행이 많거나 조인 순서

다음 차시(3-5) 준비

원본 orders.csv 가 그대로인지 확인

1 **cd ~/data-eng/week3**
3-1 의 orders.csv — 결측·공백·중복을 일부러 심어 둔 원본

2 **ls -l make_orders.py orders.csv**
orders.csv 가 36244 bytes

3 **wc -l orders.csv**
1016 줄(머리글 1 + 1015행)

없으면

python3
make_orders.py

시드 고정이라 다시 만들어도 같은 원본이 나온다

다음 회차 예고

3-5 · Pandas와 DuckDB를 이용한 하이브리드 가공

01

이론 1H

하이브리드 설계
역할 분담·품질 항목

02

실습 1.5H

정제 파이프라인
함수 단위 구현

03

실습 1H

품질 리포트 작성
주차 산출물 제출

04

정리 0.5H

주차 회고
산출물 점검

준비: 3-1 의 orders.csv(36244 bytes · 1016 줄) — 없으면 make_orders.py 재실행(시드 고정)

3-4 · 정리·회고 0.5H 마무리

다음 회차는 3-5 하이브리드 가공

DuckDB 로 추출하고 Pandas 로 정제하는 파이프라인을 만들어 품질 리포트와 정제 Parquet 을 제출합니다

✓ Pandas·DuckDB 역할 분담 설계

✓ 결측·중복·타입 오염 품질 지표 정의

✓ 재실행 가능한 정제 함수

✓ 품질 리포트 + 정제 Parquet 제출

준비할 것

3-1 의 orders.csv · venv 활성화